AI Capstone HW5 Report

Group 08

1. Reposity Contents

The repository follows the required submission structure specified in Section 16.3.

In this project, we primarily utilized **Protégé** for ontology construction, reasoning, and SPARQL query execution. To complement these native features, we implemented a **Python-based** workflow to perform additional reasoning and queries, generating well-structured and highly readable tables. Furthermore, we integrated **Widoco** to enhance the overall documentation and readability of our ontology.

1-1. Ontologys (ontology/)

This folder contains the main authored ontology and all required ontology-related artifacts.

- `group-ontology.ttl`

The main ontology developed by Group 08.

It's authored in Protégé and exported as a .ttl file.

- `inferred-results-protege.ttl`

The output graph generated after OWL reasoning.

Contains two inferred class memberships, cap:GraspableObject and g08:ColorSortableObject.

It's generated from Protégé and exported as a .ttl file.

- `inferred-results-python.ttl`

The output graph generated from `run_reasoning.py`.

- `ontology/imports/course-affordance.ttl`

The required shared vocabulary provided by the course.

1-2. Source Code (src/)

- `run_reasoning.py`

In addition to using Protégé's built-in reasoner and SPARQL plugin, we implemented a Python script to

automate the reasoning and SPARQL query process. This allowed us to generate a more clearly classified and structured table of the results.

1-3. SPARQL Queries (queries/)

This folder contains SPARQL queries used to evaluate and validate the ontology under different reasoning and analysis purposes.

- `queries/graspable_objects.rq`

This is the required baseline query for the assignment.

It retrieves all inferred cap:GraspableObject individuals after reasoning.

This query is used in both Protégé and the Python reasoning pipeline for verification of inference results.

- `queries/color_sortable_objects.rq`

This query is designed for the group's extended task involving color-based sorting.

It retrieves individuals classified under the extended concept of color-sortable objects.

This query is primarily executed in Protégé for inspection and validation of the advanced ontology design.

- `queries/task_objects.rq`

This query is used within the Python reasoning workflow.

It serves as a preprocessing step to filter relevant task-related individuals before generating final output tables.

This query is not primarily used in Protégé, but supports automated pipeline execution in run_reasoning.py.

1-4. Query and Widoco Results (results/)

This folder stores outputs generated from executing SPARQL queries under reasoning-enabled conditions.

- `protege_graspable_objects_output.txt`
- `protege_graspable_objects_output_screenshot.png`
- `python_graspable_objects_output.txt`

Output of baseline requirement graspable_objects.rq.

Used to verify inferred instances of cap:GraspableObject.

- protege_color_sortable_objects_output.txt
- protege_color_sortable_objects_output_screenshot.png

Output of color_sortable_objects.rq.

Used to validate the extended color-sorting reasoning results in Protégé.

- python_task_objects_output.txt

Output of run_seasoning.py.

Used internally by the Python pipeline for constructing final structured result tables.

- widoco_screenshot_crossref.png
- widoco_screenshot_index.png

The screenshot of Widoco.

1-5. Widoco (doc/widoco/doc/)

This folder contains the ontology documentation generated using WIDOCO.

Automatically generated HTML documentation of the ontology.

Provides human-readable browsing of classes, properties, and individuals.

Includes ontology metadata, diagrams, and term descriptions.

1-6. Python Environment Requirements

- requirements.txt

Specifies Python dependencies required to execute run_reasoning.py.

2. Ontology Design Rationale

The ontology is primarily designed based on the structure and semantics defined in the provided course ontology course-affordance.ttl.

The ontology is designed to represent robot manipulation scenarios in which a robot interacts with physical objects under different task contexts.

we adopt a generic design based on four main concepts:

- Physical objects
- Affordances
- Tasks
- Task roles

This design allows the same object to participate in different tasks while taking different semantic roles.

The affordance-centered design also enables reasoning over object capabilities instead of relying solely on object categories.

3. Namespace Policy

The ontology follows a two namespaces strategy as spec requirement.

Course Ontology Namespace

`cap:`

Contains core concepts provided by the course ontology and shared across the project.

`g08:`

Contains concepts introduced by Group 08.

This separation clearly distinguishes reused concepts from group-specific extensions.

Note on Namespace: In our implementation, GraspableObject was intentionally assigned to the local group namespace (g08:GraspableObject) rather than the cap: prefix. This design choice was maintained throughout the development pipeline to preserve the integrity of our generated graphs and query results. By localizing the class under g08:, we ensure that our team's specific extensions and reasoning workflows remain modular and decoupled from the shared core vocabulary, preventing any potential namespace pollution.

4. Ontology Terms: Reused and Newly Introduced

To ensure semantic consistency and enable group-specific reasoning extensions, we strictly separate the vocabularies in our repository into **Reused Terms** (adopted from the core course ontology and

standard semantic specifications) and **Newly Introduced Terms** (classes and instances customized by Group 08).

4.1 Reused Terms

Reused terms include the foundational building blocks of the project structure, consisting of the explicit course ontology requirements and global W3C standards.

1. Core Course Ontology Classes (cap:)

We reused the pre-defined environment and task-specific concepts to construct our local world model:

Abstract & Structural Classes: cap:PhysicalObject, cap:Affordance, cap:TaskRole, cap:Basket.

Task-Specific Object Classes: cap:Cup, cap:Knife, cap:Fork, cap:Plate, cap:ToyBlock.

Affordance Profiles: cap:GraspingAffordance, cap:ContainmentAffordance, cap:StackabilityAffordance, cap:SupportAffordance.

Task Roles: cap:TargetObject, cap:ReferenceObject, cap:ContainerTarget, cap:CollectableObject.

2. Core Course Ontology Properties (cap:)

To link our physical entities with their semantic roles and perceptual observations, we reused the following properties directly:

Object Properties: cap:hasTaskRole, cap:hasAffordance, cap:hasTargetObject, cap:hasReferenceObject, cap:canBeManipulatedBy.

Datatype Properties: cap:hasColor, cap:hasObjectLabel, cap:hasPoseFrame, cap:hasApproxWidth.

3. Standard Semantic Web Vocabularies

Universal W3C terms were reused to maintain proper structural metadata and data typing:

OWL/RDF Primitives: rdf:type, rdfs:subClassOf, rdfs:label, rdfs:comment, owl:Class, owl:ObjectProperty, owl:DatatypeProperty, owl:Restriction, owl:equivalentClass, owl:intersectionOf.

Metadata & Types: dcterms:creator, dcterms:created, dcterms:description, dcterms:title, dcterms:license, vann:preferredNamespacePrefix, vann:preferredNamespaceUri, foaf:Person, foaf:name, foaf:mbox, and xsd:string, xsd:decimal, xsd:date.

4.2 Newly Introduced Terms

Newly introduced terms represent the customized reasoning extensions and concrete physical

instances generated locally under our group's namespace (g08:).

1. Group-Extended Classes (Reasoning & Task Expansion)

To support advanced automated classification via our Python pipeline and Protégé reasoners, we introduced:

g08:GraspableObject: An inferable class defined as an intersection of cap:PhysicalObject and an existential restriction on cap:hasAffordance some cap:GraspingAffordance. (Note: Managed under our local prefix for system verification and query formatting).

g08:ColorSortingAffordance: A subclass of cap:Affordance capturing an object's action capability to be sorted into a matching container based on visual color traits.

g08:ColorSortableObject: An inferable class defined as an intersection of cap:PhysicalObject and an existential restriction on cap:hasAffordance some g08:ColorSortingAffordance**.**

g08:ColorSortingBasket: A subclass of cap:Basket constrained to possess a g08:ColorSortingAffordance for color-based container routing.

2. Local Grounded Individuals

We populated the ontology with 12 specific grounded individuals to evaluate our semantic mapping and reasoning workflows under different simulation scenes:

Cutlery Task: g08:fork01 (Target), g08:knife01 (Target), g08:plate01 (Reference).

Cup Stacking Task: g08:blueCup01 (Target, Blue), g08:pinkCup01 (Reference, Pink).

Baseline Block Task: g08:baselineBasket01 (Container Target).

Color Sorting & Collection Task:

Blocks: g08:redBlock01 (Red), g08:blueBlock01 (Blue), g08:greenBlock01 (Green).

Baskets: g08:redBasket01 (Red), g08:blueBasket01 (Blue), g08:greenBasket01 (Green).

5. Key Axioms and Restrictions

To enable automated classification and semantic grounding, our ontology defines several key theological axioms and logical restrictions using OWL primitives. These rules allow the reasoner to infer implicit relationships from perceived object properties.

4.1 Equivalent Class Axioms and Existential Restrictions

We implemented Defined Classes (anonymous class expressions) using owl:equivalentClass

combined with owl:intersectionOf and owl:someValuesFrom restrictions. This allows the reasoner to dynamically classify objects based on their manifested affordances:

Inference Rule for Graspable Objects (g08:GraspableObject)

Logical Definition: A g08:GraspableObject is defined as any cap:PhysicalObject that has at least one cap:GraspingAffordance linked via the cap:hasAffordance property.

OWL Expression: cap:PhysicalObject AND (cap:hasAffordance some cap:GraspingAffordance)

Inference Rule for Color-Sortable Objects (g08:ColorSortableObject)

Logical Definition: A g08:ColorSortableObject is defined as any cap:PhysicalObject that exhibits a g08:ColorSortingAffordance via the cap:hasAffordance property.

OWL Expression: cap:PhysicalObject AND (cap:hasAffordance some g08:ColorSortingAffordance)

4.2 Subclass Axioms and Property Domain/Range Constraints

We maintained structural integrity and inheritance boundaries through explicit subclass axioms and property constraints inherited from the course specification:

Subclass Axioms:

g08:ColorSortingBasket is strictly asserted as a subclass of cap:Basket.

g08:ColorSortingAffordance is strictly asserted as a subclass of cap:Affordance.

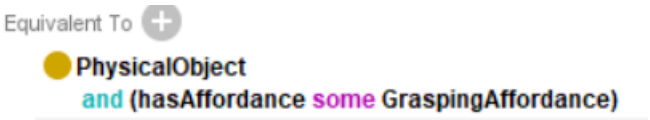
Value Restrictions:

g08:ColorSortingBasket is restricted to only receive individuals that possess a g08:ColorSortingAffordance (owl:someValuesFrom g08:ColorSortingAffordance), ensuring that color-sorting operations are constrained to appropriate target containers.

7. Reasoning Pattern

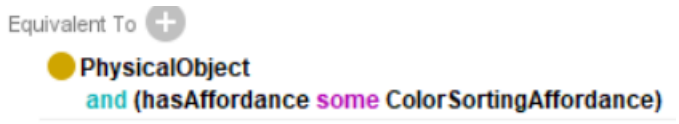
The ontology uses OWL equivalent class definitions to enable automatic classification.

GraspableObject



Any physical object possessing a grasping affordance can be automatically inferred as a graspable object.

ColorSortableObject

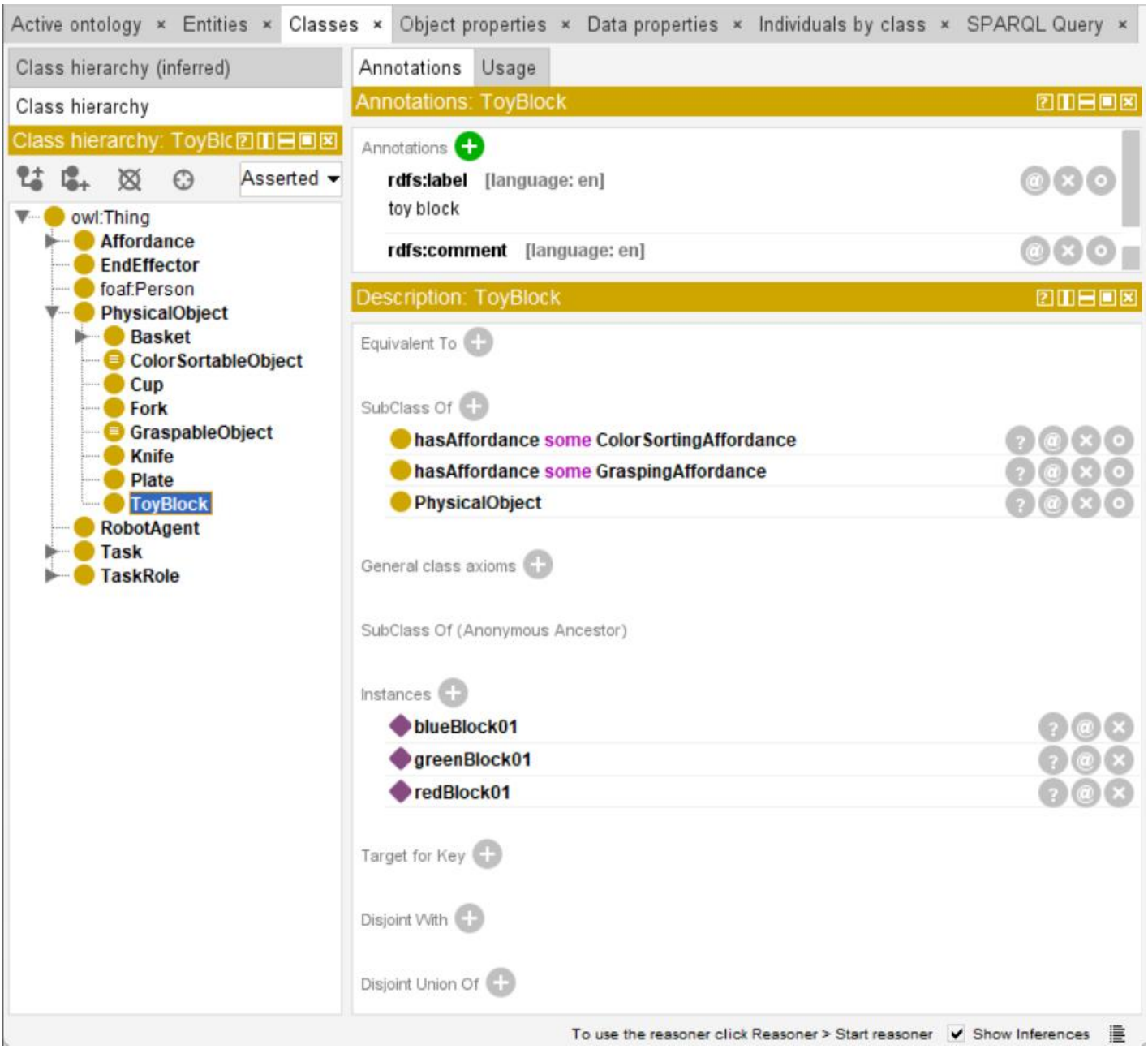


Any physical object possessing a color-sorting affordance can be automatically inferred as a color-sortable object.

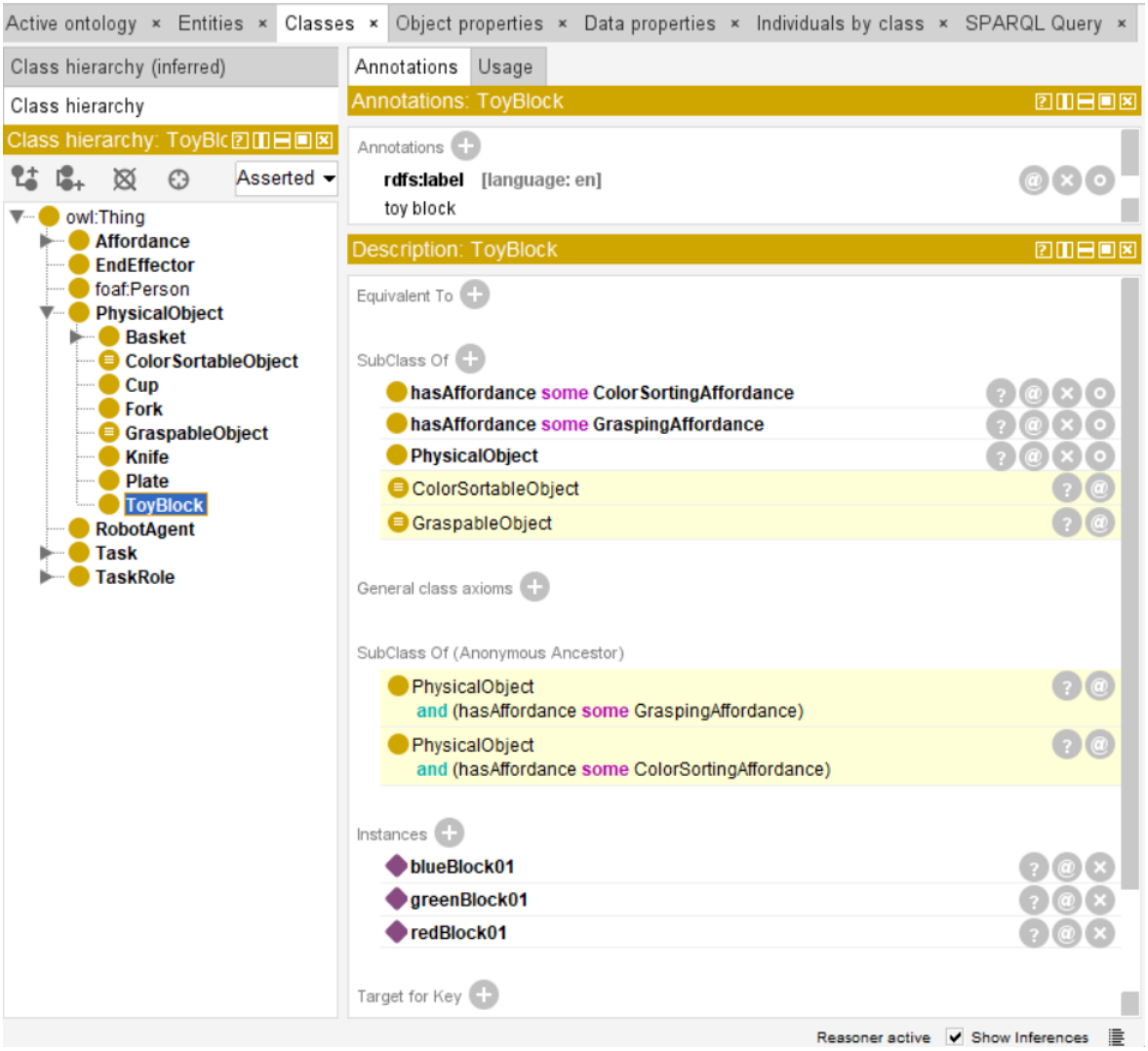
8. Reasoning Results

After running an OWL reasoner, additional class memberships are inferred automatically.

Before Reasoning (take toyBlock as an example):

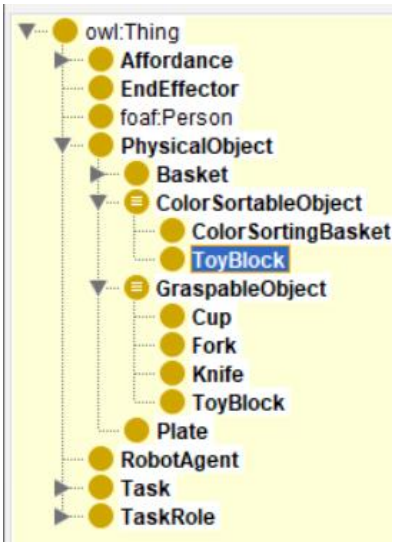


After Reasoning(Take toyblock as example):



Grasping Classification: The classes Cup, Fork, Knife, and ToyBlock were automatically classified as sub-classes of g08:GraspableObject.

Color-Sorting Classification: The classes g08:ColorSortingBasket and cap:ToyBlock were dynamically inferred under g08:ColorSortableObject.



9. Query Results

In Protégé, we use Snap SPARQL Query plug-in for query.

Snap SPARQL Query:

```
PREFIX g08: <https://hcis.io/ontology/aicapstone/2026/group08/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a g08:ColorSortableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

Execute

?obj	?label	?role
g08:blueBasket01	blue_basket^^xsd:string	cap:ContainerTarget
g08:blueBlock01	blue_block^^xsd:string	cap:CollectableObject
g08:greenBasket01	green_basket^^xsd:string	cap:ContainerTarget
g08:greenBlock01	green_block^^xsd:string	cap:CollectableObject
g08:redBasket01	red_basket^^xsd:string	cap:ContainerTarget
g08:redBlock01	red_block^^xsd:string	cap:CollectableObject

Snap SPARQL Query:

```
PREFIX g08: <https://hcis.io/ontology/aicapstone/2026/group08/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a g08:GraspableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

Execute

?obj	?label	?role
g08:blueBlock01	blue_block^^xsd:string	cap:CollectableObject
g08:blueCup01	blue_cup^^xsd:string	cap:TargetObject
g08:fork01	fork^^xsd:string	cap:TargetObject
g08:greenBlock01	green_block^^xsd:string	cap:CollectableObject
g08:knife01	knife^^xsd:string	cap:TargetObject
g08:pinkCup01	pink_cup^^xsd:string	cap:ReferenceObject
g08:redBlock01	red_block^^xsd:string	cap:CollectableObject

Result of run_reasoning.py:

The baseline graspable objects query:

```
QUERY: graspable_objects
Reasoner: owlrl (OWL RL / RDF(S) closure)
============================================================
obj                  label                role
------------------------------------------------------------
blueBlock01          blue_block           CollectableObject
blueCup01            blue_cup             TargetObject
fork01               fork                 TargetObject
greenBlock01         green_block          CollectableObject
knife01              knife                TargetObject
pinkCup01            pink_cup             ReferenceObject
redBlock01           red_block            CollectableObject

Total inferred GraspableObject instances: 7
```

The all 12 task object instances alongside their respective perceptual labels, color attributes, assigned task roles, and inferred logical classes. The boolean columns clearly validate the accuracy of our OWL RL reasoning execution

```
QUERY: task_objects
Reasoner: owlrl (OWL RL / RDF(S) closure)
================================================================================================
obj                  label                color    role                 GraspableObj   ColorSortableObj
------------------------------------------------------------------------------------------------
blueBlock01          blue_block           blue     CollectableObject    true           true
greenBlock01         green_block          green    CollectableObject    true           true
redBlock01           red_block            red      CollectableObject    true           true
baselineBasket01     baseline_basket      -        ContainerTarget      false          false
blueBasket01         blue_basket          blue     ContainerTarget      false          true
greenBasket01        green_basket         green    ContainerTarget      false          true
redBasket01          red_basket           red      ContainerTarget      false          true
pinkCup01            pink_cup             pink     ReferenceObject      true           false
plate01              plate                -        ReferenceObject      false          false
blueCup01            blue_cup             blue     TargetObject         true           false
fork01               fork                 -        TargetObject         true           false
knife01              knife                -        TargetObject         true           false

Total task object instances: 12
```

10. Design Choices

Our team made the following key design decisions during the development of the ontology to enhance modularity, clarity, and project reproducibility:

Localization of GraspableObject Namespace

We intentionally defined the GraspableObject class under our local group namespace (g08:) instead of the shared course namespace (cap:). This design choice ensures that our local reasoning rules and specific property restrictions remain modular and fully independent from the core course vocabulary,

preventing any potential namespace pollution.

Affordance-Driven Class Extension for Color Sorting

To support advanced task routing in simulation, we extended the baseline ontology by introducing a dedicated g08:ColorSortingAffordance. This choice allows the reasoner to automatically cluster objects into g08:ColorSortableObject and containers into g08:ColorSortingBasket, creating a clean logical layer for sorting tasks.

External Python Workflow for Enhanced Readability

While Protégé serves as the primary authoring tool, we chose to implement an external Python pipeline (run_reasoning.py). This automation executes the reasoning and SPARQL queries sequentially, exporting the raw graph outputs into highly structured and readable text tables for clearer evaluation.

11. Limitations

Although our ontology successfully performs automated reasoning and data classification, we have identified the following key limitations in its current design:

Static Environment Representation vs. Dynamic Physics

The current ontology structure operates as a static knowledge model. While it excels at logical classification based on fixed assertions, it cannot natively handle real-time state changes or temporal variations in a dynamic robotics simulation, such as an object changing its pose, getting hidden behind obstacles, or being moved by the robot agent during runtime.

Hardcoded Literal Matching and Lack of Semantic Flexibility

Our color-sorting reasoning chain heavily relies on exact literal matching of datatype values, such as identifying a string as red. The ontology lacks the semantic flexibility to handle ambiguous, real-world sensory inputs, such as minor color variations, deep red, or varying lighting conditions in perception. This makes the system fragile when encountering unforeseen environmental inputs without pre-defined precise naming.

12. Conclusion

In conclusion, our group has successfully constructed, extended, and validated a robust ontology framework for the AI Capstone 2026 project. By combining the core course requirements with our group's customized g08 namespace extensions, we developed a functional world model capable of semantic reasoning and automated object classification.

The deployment of the OWL reasoner perfectly confirmed our logic design, successfully inferring implicit classes such as GraspableObject and ColorSortableObject based on explicit physical affordances. Furthermore, our implementation of an external Python-based reasoning pipeline and WIDOCO documentation successfully bridged the gap between complex raw semantic graphs and human-readable evaluation data. Despite minor limitations regarding real-time environment dynamics, this project effectively demonstrates the practical power of using semantic technologies to ground physical task roles and manipulation affordances in robotic task planning.

13. Additional Issue

Technical Workaround on Object Property Constraints

During development, we encountered a strict semantic constraint regarding owl:ObjectProperty in OWL 2 description logic. By definition, an owl:ObjectProperty must exclusively link two individuals (instances) rather than linking an individual to a class. However, the initial specification required relating physical objects to their respective abstract roles, such as associating a fork instance with the TargetObject role.

To resolve this syntax conflict without violating semantic web standards, we implemented a practical engineering workaround. We explicitly declared empty individuals (or dummy instances) that share the same identifiers as the abstract classes, such as creating a cap:TargetObject individual under the cap:TargetObject class. This allows properties like cap:hasTaskRole to successfully establish relationships between physical object instances and role instances. This structural adjustment satisfies the rigid requirements of the course specification while maintaining full compatibility with the Protégé authoring environment and OWL reasoners.

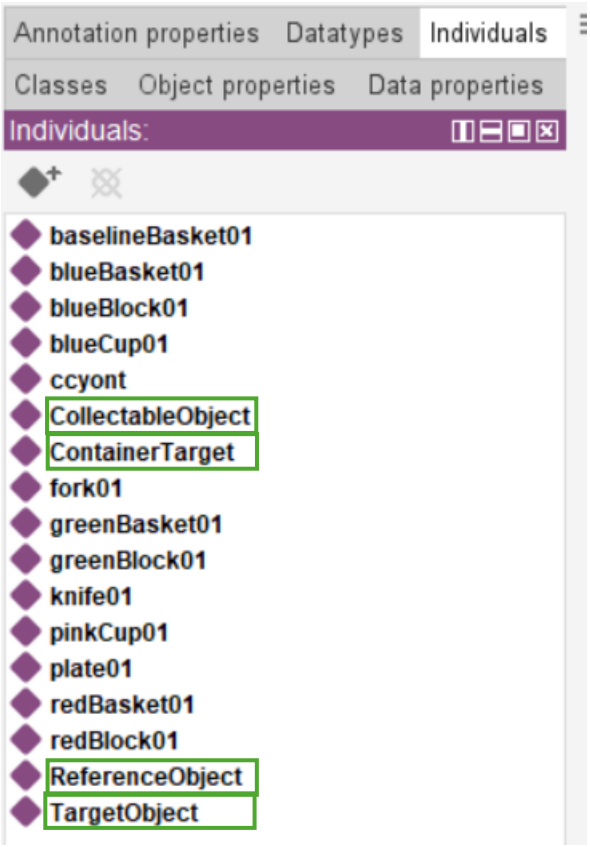
```
#################################################################
#    Individuals
#################################################################

###  https://hcis.io/ontology/aicapstone/2026/CollectableObject
cap:CollectableObject rdf:type owl:NamedIndividual ,
                              cap:CollectableObject .


###  https://hcis.io/ontology/aicapstone/2026/ContainerTarget
cap:ContainerTarget rdf:type owl:NamedIndividual ,
                             cap:ContainerTarget .


###  https://hcis.io/ontology/aicapstone/2026/ReferenceObject
cap:ReferenceObject rdf:type owl:NamedIndividual ,
                             cap:ReferenceObject .


###  https://hcis.io/ontology/aicapstone/2026/TargetObject
cap:TargetObject rdf:type owl:NamedIndividual ,
                          cap:TargetObject .
```


Annotation properties
Datatypes
Individuals
Classes
Object properties
Data properties
Individuals:
baselineBasket01
blueBasket01
blueBlock01
blueCup01
ccyont
CollectableObject
ContainerTarget
fork01
greenBasket01
greenBlock01
knife01
pinkCup01
plate01
redBasket01
redBlock01
ReferenceObject
TargetObject